

SiguoZero

四暗 / 双明 / 二人军棋

Transformer

双模型自对弈强化学习系统

只使用可执行军棋规则、玩家可见信息和最终胜负；不使用人类棋谱、专家标签、价值塑形或显式 belief head。

核心约束	已采用的实现
动作路径	起点编码与终点编码的有向二元组
玩家模型实例	每个模式仅 1 Policy + 1 Layout；二人/四国全部座位共享，绝不按座位复制
策略采样	冻结旧策略选 4 个根路径；每根 2 次终局模拟；所有后缀继续按旧策略采样
时序输入	初始棋盘 token 永久保留 + 最近最多 1000 个动作后棋盘 token
回报	只在终局返回本队 +1、和棋 0、失败 -1；连续 60 步无子力交互为和棋
确定知识	候选组合唯一性判定；存活身份写棋盘；阵亡按每名对手 25 位记录
可靠恢复	update 0 起原子保存；中盘、逐玩家身份/阵亡先验、历史、优化器及 RNG 均可恢复

架构说明书 v1.4 · 2026-09-03

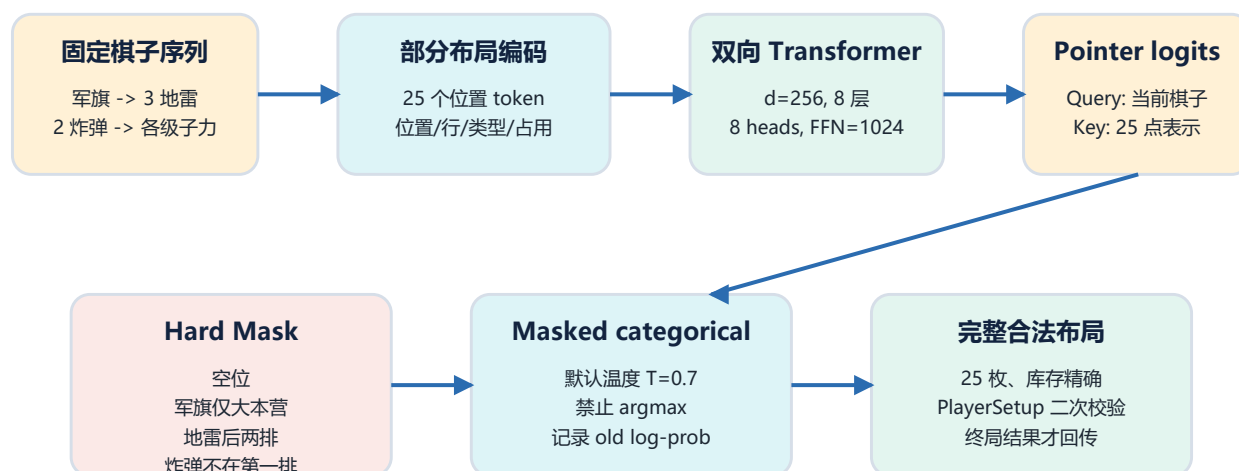
1. 系统边界与三种训练模式



模式	人数 / 棋盘	主视角可见身份	独立运行目录
四暗 four_dark	4 人 / 129 点	己方精确; 其余未确定时为 1/2/3	runs/four_dark
双明 double_open	4 人 / 129 点	己方精确; 对家为 94..105; 敌方未定为 1/3	runs/double_open
二人 two_player	2 人 / 60 点	己方 30..41; 对手未定为 2, 确定后 94..105	runs/two_player

三种模式共享同一套经过测试的组件, 但训练入口、日志、检查点、优化器和 reference 模型完全分离。每个模式只创建一个当前 Policy 和一个当前 Layout, 所有座位仅维护各自主视角状态并调用同一对象; 参数量和玩家显存绝不乘以人数。训练另有一份只用于 KL 的 reference 模型对, 它不控制玩家。加载检查点时校验模式, 禁止跨模式误用。

2. 布局模型：棋子条件 Pointer Decoder



重复 25 次：选择位置 -> 更新部分布局 -> 下一枚固定棋子

固定库存与顺序

25 步固定序列为：军旗；地雷 x3；炸弹 x2；司令 x1；军长 x1；师长/旅长/团长/营长各 x2；连长/排长/工兵各 x3。固定棋子、只预测位置，使数量约束天然成立。

模块	bootstrap 设置	输出 / 约束
位置 token	d_model=256	位置、行、是否大本营、当前占用、玩法模式
布局 Transformer	8 层、8 heads、SwiGLU FFN=1024	25 点双向上下文 + pooled layout token
Pointer query	当前棋子 + 步号 + 模式 + pooled 状态	对 25 个位置产生 logits
生成	temperature=0.7 categorical	Hard Mask 后采样并保存每步 old log-prob

布局轨迹在游戏结束前不得获得胜负梯度；完整布局始终再由 PlayerSetup 权威校验。

3. 策略输入：动作与完整棋盘共同构成 token



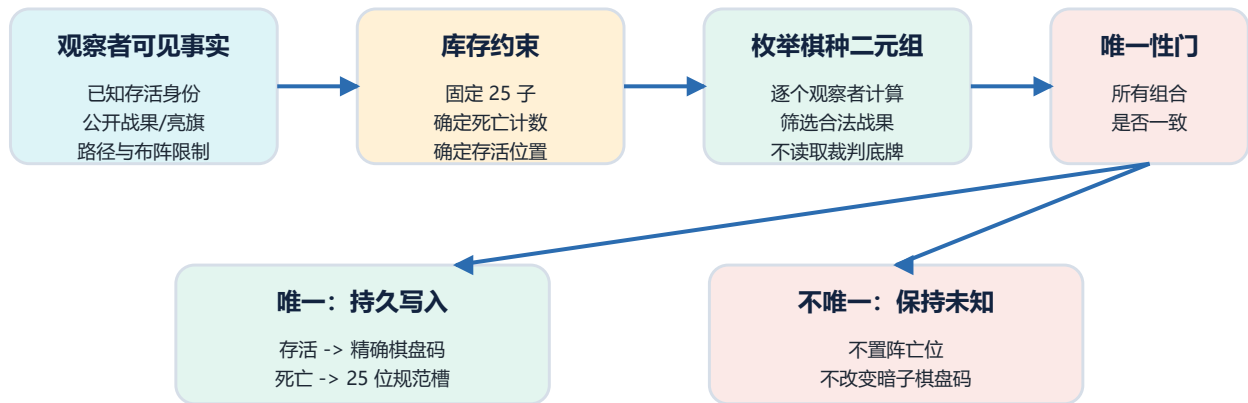
$$T_0 = [\mathbf{0}_{256}; g_0], \quad T_t = [q_t; g_t] \in \mathbb{R}^{512}$$

棋盘整数码是类别 ID，不是连续军阶数值

范围	含义	示例
0	空点	0
1 / 2 / 3	四国主视角的上家 / 对家 / 下家暗子; 二人只用 2	非己方未知身份
30..41	我方精确身份	30 军旗, 31 炸弹, 32 工兵, ..., 40 司令, 41 地雷
62..73	上家已经确定的精确身份	64 工兵, 72 司令, 73 地雷
94..105	对家已经确定的精确身份; 含双明盟友及二人对手	96 工兵, 104 司令, 105 地雷
126..137	下家已经确定的精确身份	128 工兵, 136 司令, 137 地雷

精确码公式为 $\text{base}(\text{piece}) + 32 \times \text{relative-seat-block}$ 。每名其他玩家另有与布局生成顺序一致的 25 位确定阵亡行：四暗 75 位，双明只含两敌 50 位并由盟友共享，二人 25 位。候选战斗组合只有在全部合法解释一致时才写入身份/阵亡结论；不读取裁判暗子筛选，非唯一猜测保持未知。

3.1 确定阵亡先验与无需猜测的死规则



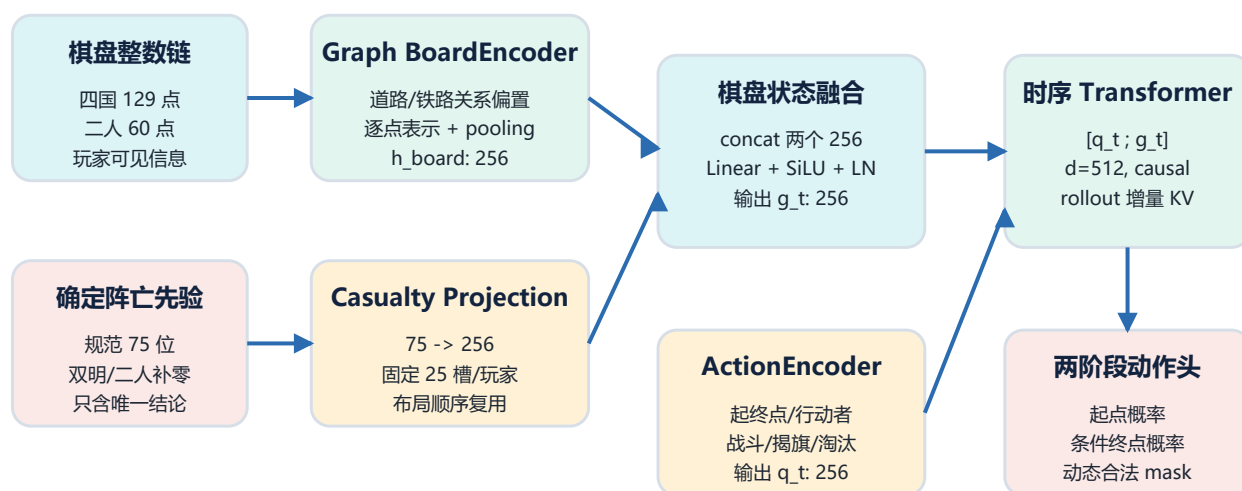
双明：自己与对家盟友共享确定知识；四暗：私有结论严格隔离

$$D_t^{i,p} \in \{0, 1\}^{25}, \quad \dim(D_t^i) = 75 \text{ (four_dark)}, 50 \text{ (double_open)}, 25 \text{ (two_player)}$$

公开条件	唯一结论
司令死亡	军旗亮出；司令死亡槽置 1
工兵进攻存活且未夺旗	守方死亡子只能是地雷
双方移除且仅一方新亮旗	新亮旗方司令，另一方炸弹；双方新亮旗则双方司令
亮旗方存活棋子吃已知师长	其司令已死，进攻子只能是军长
已知普通军阶同归，守子未移动且在本阵首排	首排排除炸弹，守子只能是同军阶
军旗被扛 / 玩家公开出局	该方规范 25 位全部置 1

同归仍可能有多个棋种解释时不置位；确定阵亡只记录先验事实，不记录概率推断。

4. 策略模型：图棋盘编码 + 因果时序 Transformer



阵亡先验融合不增加序列长度；动作概率仍由起点与条件终点分布组成

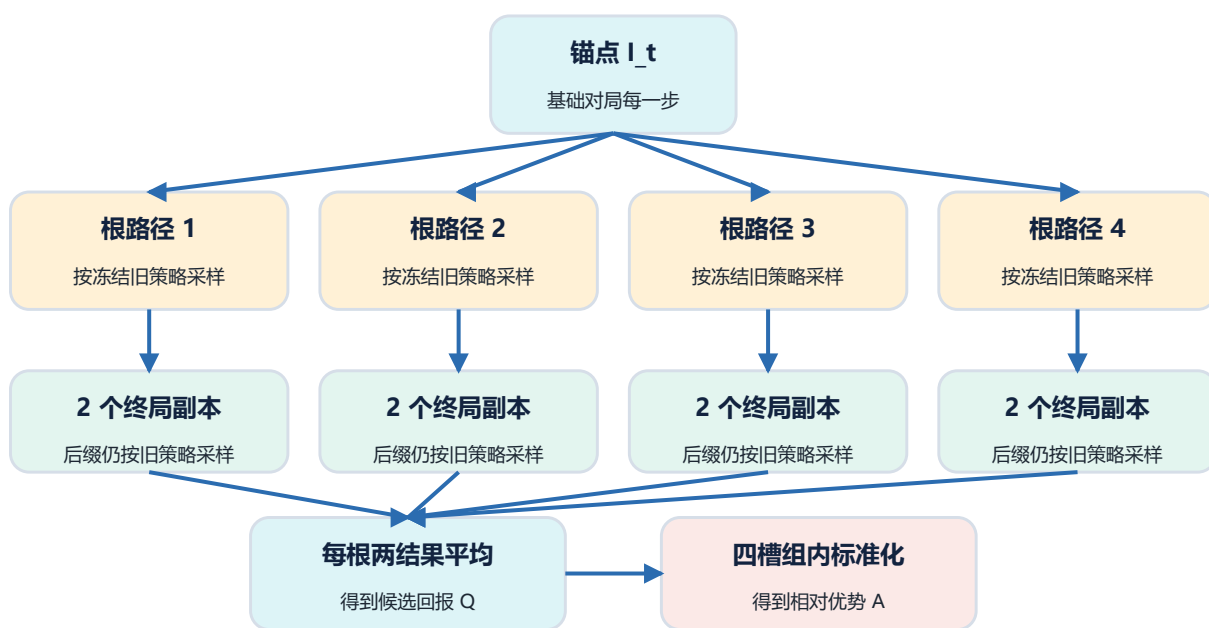
$$h_t^D = W_D \tilde{D}_t, \quad g_t = \text{LN}(\text{SiLU}(W_G[h_t^B; h_t^D] + b_G)) \in \mathbb{R}^{256}$$

$$\log \pi_{\theta}(a \mid I) = \log \pi_{\theta}(s^{\text{src}} \mid I) + \log \pi_{\theta}(s^{\text{dst}} \mid s^{\text{src}}, I)$$

层级	bootstrap	main	extended
BoardEncoder	4 层, d=256	8 层, d=256	12 层, d=256
时序 Transformer	8 层, d=512	32 层, d=512	48 层, d=512
Layout	8 层, d=256	16 层, d=256	24 层, d=256
Policy 参数	26,610,696	144,157,704	215,534,600
Layout 参数	8,887,296	17,292,288	25,697,280
模型对合计	35,497,992	161,449,992	241,231,880

75 位阵亡投影与棋盘融合只增加 151,040 个 Policy 参数，不改变 512 维时序宽度。所有动作都经动态合法 mask 后概率采样；表中参数只计一份共享 current，不按四个座位乘四。

5. Game-GRPO: 4 条根路径, 每条模拟 2 次



每个锚点固定 $4 \times 2 = 8$ 条终局续局；不穷举合法动作，也不递归展开整棵搜索树。

终局回报与优势

$$Q_{b,k} = \frac{R_{b,k,1} + R_{b,k,2}}{2}, \quad \mu_b = \frac{1}{4} \sum_{j=1}^4 Q_{b,j}, \quad \sigma_b = \sqrt{\frac{1}{4} \sum_{j=1}^4 (Q_{b,j} - \mu_b)^2}$$

$$A_{b,k} = \frac{Q_{b,k} - \mu_b}{\sigma_b + \varepsilon_A}, \quad \varepsilon_A = 10^{-4}$$

每个终局只返回本队胜 1、和 0、负 -1。同一根动作的两个结果先平均，再在四个候选槽内标准化；若四个候选回报完全相同，则该锚点优势置零，不添加中间奖励。

5.1 Game-GRPO 与双模型损失 (LaTeX 排版)

行为概率比

$$\rho_{b,k}(\theta) = \exp[\ell_{\theta}(I_b, a_{b,k}) - \ell_{\text{old}}(I_b, a_{b,k})]$$

根动作裁剪目标

$$\mathcal{L}_{P, \text{GRPO}} = -\frac{1}{4B} \sum_{b=1}^B \sum_{k=1}^4 \min(\rho_{b,k} A_{b,k}, \text{clip}(\rho_{b,k}, 1 - \varepsilon, 1 + \varepsilon) A_{b,k})$$

带阶段参考与熵正则的完整策略损失

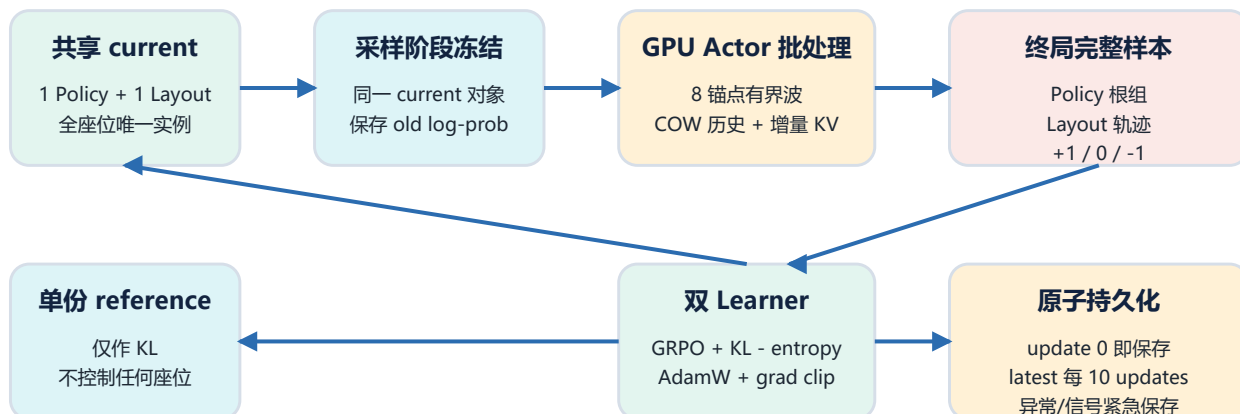
$$\mathcal{L}_P = \mathcal{L}_{P, \text{GRPO}} + \beta_P D_{\text{KL}}(\pi_{\theta} \| \pi_{P, \text{ref}}) - \alpha_P \mathcal{H}(\pi_{\theta})$$

布局 Pointer 的终局损失

$$\mathcal{L}_{L, \text{GRPO}} = -\frac{1}{25BG_L} \sum_{b,k,t} \min(\rho_{b,k,t}^L A_{b,k}^L, \text{clip}(\rho_{b,k,t}^L, 1 - \varepsilon_L, 1 + \varepsilon_L) A_{b,k}^L)$$

候选槽已经按旧策略有放回采样，损失中不能再次乘旧策略概率。策略优势只回传根动作；后缀动作仅负责把分支采样到终局。布局的 25 个 Pointer 决策共享对应座位的终局优势，但不使用 25 个概率比率的乘积。

6. 训练闭环、日志与可恢复性



恢复：模型/reference/优化器/中盘/逐玩家存活身份+阵亡库存/1000 步历史/全部 RNG

训练项	默认设置
全局 Policy batch	128 个完整锚点组 = 512 个根回报 = 1024 条终局续局
优化	AdamW; betas=(0.9,0.95); weight decay=0.05; grad clip=1.0; 最多 3 epochs
学习率	Policy 1e-4 起, 2000 updates warmup 后余弦至 1e-6; KL 超限自动减半; Layout 5e-5
正则	GRPO clip=0.2; KL beta=0.02, 目标 0.015/action; entropy alpha=0.01
混合精度	CUDA BF16, BF16 不可用时 FP16 + GradScaler; TF32 开启; 归约 FP32; activation checkpoint 开启
详细日志	JSONL + latest JSON + train.log + 可选 TensorBoard; loss/KL/熵/clip/胜和负/吞吐/显存

安全策略是 fail-closed: 非空目录没有 latest.pt 时拒绝随机重启; --no-resume 只允许空目录。正常启动自动加载 latest, 不需要额外 --resume。

7. CUDA 容量实测与训练预算

实测平台：WSL2 Ubuntu 24.04, RTX 4090 24GB, PyTorch 2.11.0+cu130。共享 current Policy/Layout 各一份及 KL reference 各一份常驻；没有四份座位模型。rev.14 的 rollout cache 只保存冻结前缀，进入 learner 前清空。

模型	microbatch	单步时间	峰值分配	24GB 默认
bootstrap	1 / 8 / 16 / 24	0.59 / 2.57 / 4.98 / 7.06 s	1.71 / 7.09 / 13.79 / 20.48 GiB	16
main 二人	8 / 16 / 24	3.54 / 6.55 / 9.23 s	7.59 / 13.53 / 19.46 GiB	24*
extended	1 / 8	1.22 / 6.49 s	5.32 / 17.72 GiB	6

累计样本预算

阶段	learner updates	锚点	终局续局	定位
冷启动验收	约 7.8K	1.0M	8M	验证规则、信息隔离、损失与吞吐
主训练	约 62.5K	8.0M	64M	建立历史回归评测和稳定棋力
顶尖容量预留	200K-600K	25.6M-76.8M	0.205B-0.614B	容量上限，不是棋力保证

分模式 H100 等效规划

模式	冷启动	主训练	顶尖容量
二人	约 2-3 GPU-days	约 9-14	约 29-128
四暗	约 9-13 GPU-days	约 58-87	约 185-834
双明	先按四暗同档	训练后按真实局长修正	训练后按真实局长修正

* 二人 main 正式任务先请求 microbatch=24；真实长局或分配碎片 OOM 时，单卡 trainer 会在尚未 step 的原子 epoch 内自动降为 12。H100/H200 microbatch 与墙钟时间必须在目标集群实测。

8. 二人/双明 RTX 4090 实测与 H100 microbatch

以下 learner 容量探针条件为最长 1001-token、BF16、activation checkpoint、共享 current 与 KL reference 常驻。当前 main Policy 为 144,157,704 参数；正式长跑按 revision 14 请求二人 microbatch=24，并保留自动降到 12 的原子 OOM 回退。时间不含规则 Actor。

模式	RTX 4090 microbatch	单步实测	峰值分配
二人	8	3.54 s	7.59 GiB
二人	16	6.55 s	13.53 GiB
二人	24	9.23 s	19.46 GiB
双明	8	5.27 s	12.68 GiB
双明	12	7.46 s	18.20 GiB

H100 80GB 的 1001-token 起测值

模型规模	二人保守	二人向上探测	双明保守	全局 128 的累积步
bootstrap	64	96	64	2
main	32	64	32	4
extended	24	32	24	6 (未批 8)

$$S_{\text{acc}} = \left\lceil \frac{B_{\text{anchor}}}{B_{\mu}} \right\rceil, \quad B_{\text{anchor}} = 128$$

microbatch 的单位是完整锚点组，不是终局 rollout。H100 数字是基于 80GB 容量和本机 4090 曲线给出的起测值，并非 H100 实测保证；启动时应从保守列开始，保留至少 10% 显存，再逐级 OOM 探测。二人棋盘只有 60 点，故单独列出可尝试的上探值；双明仍是 129 点。

H100 main 默认先用 microbatch=32；二人模式验证稳定后可试 48/64，双明不要未经实测直接超过 32。

9. rev.14 实测吞吐与 30 亿步时间

rev.14 正式第 1 个 RTX 4090 update: 1,024 条 continuation 共 327,764 步; rollout 708.10 s (462.88 步/s) , 完整 update 723.85 s, 原子检查点约 7.6 s。microbatch=24 完成 3 个 epoch、未回退, CUDA 峰值分配 7.68 GiB、缓存池 19.62 GiB。固定 256 步 A/B 从 179.6 提升到 435.4 步/s (2.42x) 。

同口径 bootstrap (35.50M 参数) 采用 actor=128 / wave=16 / KV=384: 356,068 步, rollout 551.63 s (645.48 步/s) , 完整 update 560.39 s。4090 的 30 亿步工程排期约 60-70 天; RTX PRO 6000 估算 900-1,250 步/s、约 33-48 天, 目标卡仍需实测。

$$T_{\text{days}} = \frac{3 \times 10^9 - N_{\text{done}}}{v_{\text{plies/s}} 86400 a}$$

硬件	聚合吞吐	纯 rollout	含 learner/评测规划
1× RTX 4090	463 步/s (实测)	75.0 天	85-100 天
2× RTX 4090	约 815 步/s	42.6 天	48-58 天
4× RTX 4090	约 1,389 步/s	25.0 天	29-36 天
1× RTX PRO 6000 96GB	750-1,050 步/s	33-46 天	45-60 天
2× RTX PRO 6000 96GB	1,300-1,850 步/s	19-27 天	26-38 天
1× H100 / B200 / B300	850-1,650 步/s	21-41 天	27-55 天

4090 多卡按 DDP 效率 88% (2 卡) 和 75% (4 卡) 外推。PRO/H100/B200/B300 均未在本项目实测; 更大显存允许增大 actor/learner batch, 但 Python 裁判和小 kernel 使速度不会按峰值 TOPS 线性增长。

当前仓库已支持 torchrun/DDP: 每 GPU 一个同步计算 rank, 但每个 rank 内所有座位仍共享同一 Policy/Layout。表中跨卡数值仍需目标机器完整 update 复测。

30 亿步是累计 continuation_plies, 不是 optimizer step, 也不是冠军保证。首轮平均 320.08 步/rollout, 对应约 9,153 updates; 当前循环连续运行约 77.5 天, 含可用率、分布变化和训练外评测按 2.8-3.3 个月。仍须使用至少 10 个 update 的移动中位数重算。

10. 一共训练多少个 rollout

本文把一个 rollout 严格定义为：从某个锚点固定一个根动作后，用一个独立副本继续按旧策略采样直到规则终局。每个锚点有 4 个根候选、每候选 2 个副本，因此产生 8 个 rollout。基础对局本身和训练外评测局不计入此数。

$$N_{\text{rollout}} = UB_{\text{anchor}}KM = U \times 128 \times 4 \times 2 = 1024U$$

累计阶段	learner updates	每模式 rollout	二人 + 双明合计	两模式分叉环境步
冷启动验收	约 7.8K	8.0M	16.0M	约 4.14B
有竞争力主训练	62.5K	64.0M	128.0M	约 33.09B
顶尖容量预留	200K-600K	204.8M-614.4M	409.6M-1.2288B	约 105.9B-317.6B

推荐的明确口径

问题	答案
每次 update	每个模式 1,024 个终局 rollout
二人 30 亿步	首轮折算 9.37M rollouts / 9.15K updates; 规划范围仍为 8.6M-16.4M / 8.4K-16.1K
先完成可用主训练	每模式累计 64M; 二人 + 双明一共 128M
冲击顶尖棋力	每模式累计 204.8M-614.4M; 两模式合计 409.6M-1.2288B
是否保证顶尖	不保证; 每 10K-25K updates 依据历史回归、人类盲测和熵决定继续或停止

顶尖容量是预算上限而非必须烧完。由于所有座位共享同一个模型，rollout 数不会再乘玩家数；四国四个座位只改变每条续局中的观测与行动轮转，不把 8 条续局变成 32 条。

11. 验证结果、运行入口与生产边界

验证	结果
规则/编码/共享测试	97 项 pytest 全部通过（另含 129 个参数化子例），覆盖确定身份/阵亡先验、60 步判和、暗子隔离、座位共享与 DDP
4x2 rollout	严格生成 4 个根候选 x 2 副本，全部运行到规则终局
三模式 CUDA	四暗 / 双明 / 二人分别完成训练 update、下一 update 自动恢复和检查点推理
恢复完整性	格式 v4 保存布局缓冲、中盘、逐玩家身份/阵亡库存、历史、模型/优化器/reference 与各 rank RNG
最长上下文	bootstrap/main/extended 均完成四暗 1001-token CUDA 前向、反向和 AdamW 容量探针

三个独立入口

```
python -m junqi.training.train_four_dark --device cuda
python -m junqi.training.train_double_open --device cuda
python -m junqi.training.train_two_player --device cuda

python -m junqi.training.infer_four_dark --checkpoint runs/four_dark/checkpoints/latest.pt
python tools/benchmark_cuda.py --mode four_dark --model-scale main --context-tokens 1001
--batch-size 8
python tools/benchmark_rollout.py --mode two_player --model-scale main --anchors 8
--max-game-plies 600 --full-stack
```

达到大规模预算前仍需完成

已完成：单机 DDP、增量 causal KV、持久 COW 历史、packed 棋盘、长度分桶与规则静态查表。后续：1) C++/Rust 完整批量裁判并与 Python 裁判差分；2) 自定义 paged-attention/CUDA Graph，消除 K/V stack/cat；3) 一版本 behavior snapshot 的跨 update Actor/Learner 流水；4) 训练外历史回归、人类盲测和长期断电演练。

实现与研究依据

代码：src/junqi/training/；配置：bootstrap.yaml
rev.14；资源表：docs/training_resources_zh.md。规则依据：[边锋军棋规则](#)、[军棋玩法指南](#)。硬件规格：[NVIDIA H100](#)、[NVIDIA HGX B200](#)、[RTX PRO 6000](#)。跨卡时间是项目推算，不是 NVIDIA benchmark。